# Reminder: Submission Instructions

1) First create a copy of this notebook in your drive and rename it so that it contains the course number (ISTA322), the semester (Sp22 for Spring 2022, Su22 for Summer 2022, and Fa22 for Fall 2022), the assigment code (HW1 for this assignment) and your name.

```
e.g. my copy for Spring 2023 would be ISTA_322_Sp23_HW4_dan_charbonneau)
```

2) When you are ready to submit. Prepare three files: the python file (File->Download->Download .py), the notebook file (File->Download->Download .ipynb), and **PDF** version of your notebook (**after running all cells**). You should be able to create a pdf through File -> print -> Destination: save as pdf

3) Create a new folder named firstname_lastname_hw1 (e.g my folder would be dan_charbonneau_hw4) put all three files in it. Compress (Zip) the folder you created with the files inside of it and submit this .zip file to D2L

**incorrect filenames or submission formats will result in a loss of 50% of your grade**

# HW 4 - Building a normalized RDB

The goal of this homework is to take a semi-structured non-normalized CSV file and turn it into a set of normalized tables that you then push to *your* postgres database on AWS (the one you create in Lecture 7).

The original dataset contains 100k district court decisions, although I've downsampled it to only 1000 rows to make the uploads faster. Each row contains info about a judge, their demographics, party affiliation, etc. Rows also contain information about the case they were deciding on. Was it a criminal or civil case? What year was it? Was the direction of the decision liberal or conservative?

While the current denormalized format is fine for analysis, it's not fine for a database as it violates many normalization rules. Your goal is to normalize it by designing a simple schema, then wrangling it into the proper dataframes, then pushing it all to AWS.

For the first part of this assignment you should wind up with three tables. One with case information, one with judge information, and one that has casetype information. Each table should be reduced so that there are not then repeating rows, and primary keys should be assigned within each. These tables should be called 'cases', 'judges', and 'casetype'.

designed within each. These tables should be called `cases`, `judges`, and `casetypes`.

For the last part you should make a rollup table that calculates the percent of liberal decisions for each party level and each case category. This will allow for one to get a quick look at how the political party affiliation of judges impacts the direction of a decision for different case categories (e.g. criminal, civil, labor).

**Submission** 1) Run all cells. 2) Create a directory with your name. 3) Create a pdf copy of your notebook. 4) Download .py and .ipynb of the notebook. 5) Put all three files in it. 6) Zip and submit.

## Q1 Bring in data, explore, make schema - 3 point

Start by bringing in your data to `cases`. Call a `.head()` on it to see what columns are there and what they contain.

```
## Q1 Your code starts here
import pandas as pd
cases = pd.read_csv('https://docs.google.com/spreadsheets/d/1AWLK06JOlSKImgoHNTbj7oXR5mRf
```

```
# head of cases
cases.head()
## Q1 Your function ends here - Any code outside of these start/end markers won't be grad
```

| | judge_name | party_name | gender_name | race_name | case_id | case_year | casetype_id |
|---|---|---|---|---|---|---|---|
| **0** | Thompson, Myron H. | Democrat | male | African-American/ black | 28321332 | 2011 | 3 |
| **1** | Shoob, Marvin A. | Democrat | male | white/ caucasian | 18110669 | 1993 | 14 |
| **2** | Bua, Nicholas J. | Democrat | male | white/ caucasian | 15660871 | 1983 | 2 |
| **3** | Kovachevich, Elizabeth | Republican | female | white/ caucasian | 17770934 | 1991 | 2 |
| **4** | Gilliam, Earl B. | Independent/ Other/ Unknown | male | white/ caucasian | 26621195 | 2009 | 6 |

### Make schema

OK, given that head, you need to make three related tables that will make up a normalized

OK, given that head, you need to make three related tables that will make up a normalized database. Those tables are 'cases', 'judges', and 'casetype'. If it's not clear what info should go into each, explore the data more.

*For each of the tables you create, keep the original column names from the imported cases file above. I'll be using these to test your tables, so if they don't match I won't be able to test them and you'll lose points*

Remember, you might not have keys, will need to reduce the rows, select certain columns, etc. There isn't a defined path here.

***Include an image file of your schema in your zip file in order to get the 3 points***

## Q2 Make cases table. - 6 points

Start by making a table that contains just each case's info. I would call this table that you're going to upload `cases_df` so you don't overwrite your raw data.

This table should have eight columns and 1000 rows.

Note, one of these columns should be a judge_id that links to the judges table. You'll need to make this foreign key.

Also, you can leave 'category_name' in this table as well as its id. Normally you'd split that off into it's own table as well, but you're already doing that for casetype which is enough for now.

**At this point in the semester, we've talked a lot about exploring your data and using appropriate datatypes, so my expectation going forwards is that students will take care in choosing and setting datatypes for their data.**

**Be very careful as you work through this assignment to make sure your data is set up correctly**

```
## Q2 part 1 Your code starts here
# Make judge_id in cases
cases['judge_id'] = cases['judge_name'].astype('category').cat.codes
cases.head()
```

| | judge_name | party_name | gender_name | race_name | case_id | case_year | casetype_id |
|---|---|---|---|---|---|---|---|
| **0** | Thompson, Myron H. | Democrat | male | African-American/ black | 28321332 | 2011 | 3 |
| **1** | Shoob, Marvin A. | Democrat | male | white/ caucasian | 18110669 | 1993 | 14 |
| **2** | Bua, Nicholas J. | Democrat | male | white/ caucasian | 15660871 | 1983 | 2 |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **3** | Kovachevich, Elizabeth | Republican | female | white/ caucasian | 17770934 | 1991 | 2 |
| **4** | Gilliam, Earl B. | Independent/ Other/ Unknown | male | white/ caucasian | 26621195 | 2009 | 6 |

Next steps: Generate code with `cases` View recommended plots New interactive sheet

```
# select necessary columns to make cases_df
cases_df = cases[['case_id', 'case_year', 'casetype_id', 'casetype_name', 'category_id',


# Show the head of cases_df and print it's shape?
print(cases_df.shape)
cases_df.head()
## Q2 part 1 Your function ends here - Any code outside of these start/end markers won't
```

```
(1000, 8)
```

| | case_id | case_year | casetype_id | casetype_name | category_id | category_name | judge |
|---|---|---|---|---|---|---|---|
| **0** | 28321332 | 2011 | 3 | criminal court motions | 1 | Criminal Justice Cases | |
| **1** | 18110669 | 1993 | 14 | free of religion | 2 | Civil Liberties/ Rights Cases | |
| **2** | 15660871 | 1983 | 2 | habeas corpus-state | 1 | Criminal Justice Cases | |
| | | | | habeas corpus- | | Criminal Justice | |

## Make cases table in your database

Put the helper functions (*get_conn_cur()*, *get_table_names()*, etc. from previous NB and HW) to create the connection here. Once you do that you'll need to do the following

- Connect, make a table called 'cases' with the correct column names and data types. Be sure to execute and commit the table.
- Make tuples of your data
- Write a SQL string that allows you to insert each tuple of data into the correct columns
- Execute the string many times to fill out 'cases'
- Commit changes and check the table.

I'm not going to leave a full roadmap beyond this. Feel free to add cells as needed to do the above

above.

```
## Q2 part 2 Your code starts here
import psycopg2
def get_conn_cur(): # define function name and arguments (there aren't any)
  # Make a connection
  conn = psycopg2.connect(
    # Enter the credentials for your RDB (the one you created in Lecture 7).
    # DO NOT USE THE COURSE RDB FROM PREVIOUS LECTURES. DOING SO WILL RESULT IN AN AUTOMA
    host= 'ista322cougarbellingerdb.chg62e6gampt.us-east-2.rds.amazonaws.com',
    database= 'ista322cougarbellingerDB',
    user= 'cougarbellinger',
    password= 'Bonded4-Cupcake2-Rely5-Agile4',
    port='5432')

  cur = conn.cursor()   # Make a cursor after

  return(conn, cur)   # Return both the connection and the cursor


### This is an extra function I'm giving you that allows you to drop tables from your RDB
# If you try creating the same table when it already exists on your RDB, you'll get an er
# I recommend calling this function one line above your code creating your table. eg for
def my_drop_table(tab_name):
  conn, cur = get_conn_cur()
  tq = """DROP TABLE IF EXISTS %s CASCADE;""" %tab_name
  cur.execute(tq)
  conn.commit()

def run_query(query_string):
  conn, cur = get_conn_cur() # get connection and cursor
  cur.execute(query_string) # executing string as before
  my_data = cur.fetchall() # fetch query data as before
  # here we're extracting the 0th element for each item in cur.description
  colnames = [desc[0] for desc in cur.description]
  cur.close() # close
  conn.close() # close
  return(colnames, my_data) # return column names AND data

def sql_head(table_name, n=5):
    query = f"SELECT * FROM {table_name} LIMIT {n};"
    colnames, my_data = run_query(query)
    df = pd.DataFrame(my_data, columns=colnames)
    return df

# Column name function for checking out what's in a table
def get_column_names(table_name): # arguement of table_name
  conn, cur = get_conn_cur() # get connection and cursor

  # Now select column names while inserting the table name into the WERE
  column_name_query =  """SELECT column_name FROM information_schema.columns
```

```
        WHERE table_name = '%s' """ %table_name

    cur.execute(column_name_query) # exectue
    my_data = cur.fetchall() # store

    cur.close() # close
    conn.close() # close

    return(my_data) # return

# Check table_names
def get_table_names():
    conn, cur = get_conn_cur() # get connection and cursor

    # query to get table names
    table_name_query = """SELECT table_name FROM information_schema.tables
        WHERE table_schema = 'public' """

    cur.execute(table_name_query) # execute
    my_data = cur.fetchall() # fetch results

    cur.close() #close cursor
    conn.close() # close connection

    return(my_data) # return your fetched results


conn, cur = get_conn_cur()
my_drop_table('cases')
create_cases_table = """CREATE TABLE IF NOT EXISTS cases (
    case_id INT PRIMARY KEY,
    case_year INT,
    casetype_id INT,
    casetype_name VARCHAR(255),
    category_id INT,
    category_name VARCHAR(255),
    judge_id INT,
    libcon_id INT)"""
cur.execute(create_cases_table)
conn.commit()

cases_tuples = list(cases_df.itertuples(index=False, name=None))
insert_cases = """
    INSERT INTO cases (case_id, case_year, casetype_id, casetype_name, category_id, categ
    VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
"""
cur.executemany(insert_cases, cases_tuples)
conn.commit()

conn.close()
cur.close()
```

```
cur.close()

# Use sql_head to check cases
sql_head(table_name='cases')
## Q2 part 2 Your function ends here - Any code outside of these start/end markers won't
```

| | case_id | case_year | casetype_id | casetype_name | category_id | category_name | judge |
|---|---|---|---|---|---|---|---|
| **0** | 28321332 | 2011 | 3 | criminal court motions | 1 | Criminal Justice Cases | |
| **1** | 18110669 | 1993 | 14 | free of religion | 2 | Civil Liberties/ Rights Cases | |
| **2** | 15660871 | 1983 | 2 | habeas corpus-state | 1 | Criminal Justice Cases | |
| | | | | habeas corpus- | | Criminal Justice | |

## Q3 Make judges - 6 points

Now make your judges table from the original `cases` dataframe (not the SQL table you just made).

Judges should have five columns, including the `judge_id` column you made. There should be 553 rows after you drop duplicates (remember that judges may have had more than one case).

After you make the dataset go and push to a SQL table called 'judges'.

```
## Q3 Your code starts here
#Your answer
judges_df = cases[['judge_id','judge_name', 'party_name', 'gender_name', 'race_name']].co
judges_df = judges_df.drop_duplicates()
judges_tuples = list(judges_df.itertuples(index=False, name=None))


#Run this cell
conn, cur = get_conn_cur()

my_drop_table('judges')
create_judges_table = """CREATE TABLE IF NOT EXISTS judges (
    judge_id INT PRIMARY KEY,
    judge_name VARCHAR(255),
    party_name VARCHAR(255),
    gender_name VARCHAR(255),
    race_name VARCHAR(255)
    )"""
cur.execute(create_judges_table)
conn.commit()

insert_judges = """
```

```
    INSERT INTO judges (judge_id, judge_name, party_name, gender_name, race_name)
    VALUES (%s, %s, %s, %s, %s)
"""
cur.executemany(insert_judges, judges_tuples)
conn.commit()

conn.close()
cur.close()

sql_head(table_name='judges')
## Q3 Your function ends here - Any code outside of these start/end markers won't be grad
```

| | judge_id | judge_name | party_name | gender_name | race_name |
|---|---|---|---|---|---|
| 0 | 485 | Thompson, Myron H. | Democrat | male | African-American/ black |
| 1 | 453 | Shoob, Marvin A. | Democrat | male | white/caucasian |
| 2 | 56 | Bua, Nicholas J. | Democrat | male | white/caucasian |
| 3 | 274 | Kovachevich, Elizabeth | Republican | female | white/caucasian |

## Q4 Make casetype - 6 points

Go make the casetype table. This should have only two columns that allow you to link the casetype name back to the ID in the 'cases' table. There should be 27 rows as well.

```
## Q4 Your code starts here
#Your answer
casetype_df = cases[['casetype_id', 'casetype_name']].copy()
casetype_df = casetype_df.drop_duplicates()
casetype_tuples = list(casetype_df.itertuples(index=False, name=None))
casetype_df.head()
```

| | casetype_id | casetype_name |
|---|---|---|
| 0 | 3 | criminal court motions |
| 1 | 14 | free of religion |
| 2 | 2 | habeas corpus-state |
| 4 | 6 | alien petitions |
| 5 | 19 | environmental protection |

Next steps: Generate code with casetype_df View recommended plots New interactive sheet

steps:

```
#run this cell
conn, cur = get_conn_cur()

my_drop_table('casetype')
create_casetype_table = """CREATE TABLE IF NOT EXISTS casetype (
    casetype_id INT PRIMARY KEY,
    casetype_name VARCHAR(255)
    )"""
cur.execute(create_casetype_table)
conn.commit()

insert_casetype = """
    INSERT INTO casetype (casetype_id, casetype_name)
    VALUES (%s, %s)
"""
cur.executemany(insert_casetype, casetype_tuples)
conn.commit()

sql_head(table_name='casetype')
## Q4 Your function ends here - Any code outside of these start/end markers won't be grad
```

| | casetype_id | casetype_name |
|---|---|---|
| **0** | 3 | criminal court motions |
| **1** | 14 | free of religion |
| **2** | 2 | habeas corpus-state |
| **3** | 6 | alien petitions |
| **4** | 19 | environmental protection |

## Q5 A quick test of your tables - 3 point

Below is a query to get the number of unique judges that have ruled on criminal court motion cases. You should get a value of 119 as your return if your database is set up correctly!

```
## Nothing to code here! Just run this and, if it returns 119 you should get full points!
run_query("""SELECT COUNT(DISTINCT(judges.judge_id)) FROM cases
    JOIN judges ON cases.judge_id = judges.judge_id
        WHERE casetype_id = (SELECT casetype_id FROM casetype
                  WHERE casetype_name = 'criminal court motions'); """)
```

```
(['count'], [(119,)])
```

## Q6 Make rollup table - 6 points

Now let's make that rollup table! The goal here is to make a summary table easily accessed. We're going to roll the whole thing up by the judges party and the category, but you could imagine doing this for each judge to track how they make decisions over time which would then be useful for an analytics database. The one we're making could also be used as a dimension table where we needed overall party averages.

We want to get a percentage of liberal decisions by each grouping level (party_name, category_name). To do this we need first, the number of cases seen at each level, and second, the number of liberal decisions made at each level. `cases` contains the columns `libcon_id` which is a 0 if the decision was conservative in its ruling, and a 1 if it was liberal in its ruling. Thus, you can get a percentage of liberal decisions if you divide the sum of that column by the total observations. Your `agg()` can both get the sum and count.

After you groupby you'll need to reset the index, rename the columns, then make the percentage.

Once you do that you can push to a SQL table called 'rollup'

Let's get started

```
## Q6 Your code starts here
# Make a groupby called cases_rollup. This should group by party_name and category name.



# reset your index
merged_df = pd.merge(cases_df, judges_df, on='judge_id')
cases_rollup = (merged_df.groupby(['party_name', 'category_name'])['libcon_id']
                .agg(['count', 'sum'])
                .reset_index())

print(cases_rollup.head())
```

```
                      party_name                 category_name  count  sum
     0                  Democrat  Civil Liberties/Rights Cases    218  112
     1                  Democrat        Criminal Justice Cases    107   36
     2                  Democrat        Labor & Economic Cases    126   69
     3  Independent/Other/Unknown  Civil Liberties/Rights Cases     12    5
     4  Independent/Other/Unknown        Criminal Justice Cases      8    5
```

```
print("rename your columns now. Keep the first two the same but call the last two 'total_
cases_rollup.columns = ['party_name', 'category_name', 'total_cases', 'num_lib_decisions'
print(cases_rollup.head())
```

```
     rename your columns now. Keep the first two the same but call the last two 'total_cas
```

```
Rename your columns now. Keep the first two the same but call the last two 'total_cas
                party_name                category_name  total_cases  \
0                 Democrat  Civil Liberties/Rights Cases          218
1                 Democrat        Criminal Justice Cases          107
2                 Democrat        Labor & Economic Cases          126
3  Independent/Other/Unknown  Civil Liberties/Rights Cases           12
4  Independent/Other/Unknown        Criminal Justice Cases            8

   num_lib_decisions
0                112
1                 36
2                 69
3                  5
4                  5
```

Now make a new column called 'percent_liberal'

This should calculate the percentage of decisions that were liberal in nature. Multiply it by 100 so that it's a full percent. Also use the `round()` function on the whole thing to keep it in whole percentages.

```
# make your metric called 'percent_liberal'
print("make your metric called 'percent_liberal'")
cases_rollup['percent_liberal'] = (cases_rollup['num_lib_decisions'] / cases_rollup['total_
cases_rollup['percent_liberal'] = cases_rollup['percent_liberal'].round(2)
print(cases_rollup.head())
```

```
make your metric called 'percent_liberal'
                party_name                category_name  total_cases  \
0                 Democrat  Civil Liberties/Rights Cases          218
1                 Democrat        Criminal Justice Cases          107
2                 Democrat        Labor & Economic Cases          126
3  Independent/Other/Unknown  Civil Liberties/Rights Cases           12
4  Independent/Other/Unknown        Criminal Justice Cases            8

   num_lib_decisions  percent_liberal
0                112            51.38
1                 36            33.64
2                 69            54.76
3                  5            41.67
4                  5            62.50
```

Now go and push the whole thing to a table called 'rollup'

There should be five columns and nine rows.

```
# push the whole thing to a table called 'rollup'
my_drop_table('rollup')
create_rollup_table = """CREATE TABLE IF NOT EXISTS rollup (
    party_name VARCHAR(255),
```

```
    category_name VARCHAR(255),
    total_cases INT,
    num_lib_decisions INT,
    percent_liberal FLOAT
    )"""
conn, cur = get_conn_cur()
cur.execute(create_rollup_table)
conn.commit()

cases_rollupp_tuples = list(cases_rollup.itertuples(index=False, name=None))
insert_rollup = """
    INSERT INTO rollup (party_name, category_name, total_cases, num_lib_decisions, percent_
    VALUES (%s, %s, %s, %s, %s)
    """
cur.executemany(insert_rollup, cases_rollupp_tuples)
conn.commit()

conn.close()
cur.close()



# Run this cell
sql_head('rollup')
## Q6 Your function ends here - Any code outside of these start/end markers won't be grad
```

| | party_name | category_name | total_cases | num_lib_decisions | percent_liberal |
|---|---|---|---|---|---|
| **0** | Democrat | Civil Liberties/ Rights Cases | 218 | 112 | 51.38 |
| **1** | Democrat | Criminal Justice Cases | 107 | 36 | 33.64 |
| **2** | Democrat | Labor & Economic Cases | 126 | 69 | 54.76 |